

■ SCM 2.0

Smart Contact Manager

Interview Preparation Guide

● Spring Boot ● Spring Security ● OAuth2 ● MySQL ● AWS

6
Sections

20
Q&A

8
Tech Stack
Items

4
Feature
Deep-Dives

A comprehensive, structured guide to confidently explain and defend your **Smart Contact Manager (SCM 2.0)** project in technical interviews. Covers architecture, feature deep-dives, database design, cloud deployment, and 20 must-know Q&As — in both Hinglish and Professional English.

Spring Boot

Spring Security

OAuth2

Thymeleaf

Tailwind CSS

Cloudinary

AWS EB

MySQL

Hibernate

Table of Contents

Section 1 Project Overview — Elevator Pitch

Section 2 Technology Stack — What, Why & How

Section 3 Database Schema & Entity Relationships

Section 4 Key Feature Deep-Dives

Section 5 AWS Deployment Architecture

Section 6 Top 20 Interview Questions & Answers

SECTION 01

Project Overview

■■ Hinglish Elevator Pitch

"SCM 2.0 ek secure, cloud-enabled contact management application hai. Isme users apne contacts ko manage kar sakte hain, Google/GitHub se social login kar sakte hain, contacts ko direct emails bhej sakte hain (SMTP), contact details ko client-side se Excel format me export kar sakte hain, aur application feedback aur star ratings submit kar sakte hain. Ise maine Spring Boot, Hibernate, MySQL, Thymeleaf aur Tailwind CSS ke sath build kiya hai aur AWS Elastic Beanstalk par deploy kiya hai."

■ Professional English Response

"Smart Contact Manager (SCM 2.0) is a secure, cloud-integrated contact management system built on Spring Boot, Spring Security (OAuth2), Hibernate, and MySQL. It features a modern, responsive user interface styled with Tailwind CSS and Thymeleaf templates. The system allows users to perform full CRUD operations on contacts, upload contact avatars using the Cloudinary API, perform server-side pagination and dynamic searching, export directories to Excel instantly, and dispatch emails directly through custom SMTP integrations. Lastly, it is deployed on AWS Elastic Beanstalk using Nginx as a reverse proxy, connected to an Amazon RDS database."

SECTION 02

Technology Stack

Layer / Feature	Technology	Why?	How?
Backend Framework	Spring Boot 3.2.5	Auto-config, embedded server, DI, ecosystem	WebApplicationInitializer, SpringApplication.run()
Security Layer	Spring Security 6	Prevents SQLi, CSRF, session hijacking; SecurelyFilterChain	SecurityFilterChain bean; BCryptPasswordEncoder
Social Login	OAuth2 Client	Google/GitHub auth; zero registration friction	OAuth2SuccessHandler saves user from claims
Database & ORM	MySQL + Spring Data JPA	Relational data (User->Contacts); less boilerplate	@Entity, @OneToMany, JpaRepository
UI Templates	Thymeleaf + Tailwind CSS	Server-side binding; rapid responsive styling	th:include.html layout; Tailwind output.css
Image Hosting	Cloudinary API	Ephemeral cloud servers lose local files	MultipartFile -> Cloudinary SDK -> CDN URL
Direct Mailing	Spring Mail (SMTP)	Send verification/update emails from dashboard	JavaMailSender, Gmail port 587, MimeMessage
Excel Export	TableToExcel JS	Zero backend CPU; instant client-side download	JSON.stringify; TableToExcel.convert(#table)

SECTION 03

Database Schema & Entity Relationships

Entity Overview

USER
userId (PK)
name
email (UK)
password
about
profilePic
enabled
emailVerified
provider
CONTACT
id (PK)
name
email
phoneNumber
address
picture
cloudinaryImagePublicId
favorite
user_id (FK)
FEEDBACK
id (PK)
subject
message
rating
submissionTime
user_id (FK)
DIRECT_MESSAGE

id (PK)
recipientEmail
subject
body
sentTime
user_id (FK)

SOCIAL_LINK	
id (PK)	
link	
title	
contact_id (FK)	

Key Relationships Explained

User → Contact	@OneToMany (LAZY)		One user has many contacts. LAZY fetch — contacts not loaded when fetching user profile unless explicitly accessed.
Contact → User	@ManyToOne @JsonIgnore	+	Many contacts map to one user. @JsonIgnore prevents infinite recursion in JSON serialization.
User → Feedback	@OneToMany (LAZY)		Users submit multiple star-rating feedback entries. Lazy to avoid loading feedback on every user fetch.
User → DirectMessage	@OneToMany (LAZY)		Tracks email dispatch history as an audit trail. Lazy loaded.
Contact → SocialLink	@OneToMany (EAGER)		Social links are small and always shown with contact — Eager loading is appropriate here.

SECTION 04

Key Feature Deep-Dives

Feature 01

OAuth2 Login & Registration Flow

THE PROBLEM

Normal logins require passwords. Users prefer social logins for convenience.

- 1 User clicks Google login → request goes to /oauth2/authorization/google
- 2 Spring Security redirects to Google Authorization endpoint
- 3 User approves → Google redirects back with authorization code
- 4 Spring exchanges code for Access Token → retrieves profile claims
- 5 OAuthSuccessHandler extracts name/email/picture (or login/avatar for GitHub)
- 6 Checks DB if email exists → registers new user if not
- 7 Saves user with provider tag (GOOGLE or GITHUB) → redirects to /user/profile

Feature 02

Image Uploading via Cloudinary SDK

THE PROBLEM

Local server storage is lost on cloud redeployment (AWS EB clears disk). Multiple scaled instances won't share local files.

- 1 Controller receives image as MultipartFile
- 2 Reads bytes and calls: cloudinary.uploader().upload(data, ObjectUtils.asMap("public_id", filename))
- 3 Applies transformations: width(150), height(150), crop("fill") before saving
- 4 Returns CDN URL → saved to Contact.picture field in MySQL
- 5 Ensures consistent layout and fast global delivery

Feature 03

Server-Side Pagination & Null-Safe Searching

THE PROBLEM

Loading thousands of contacts at once creates high JVM memory usage and slow queries.

- 1 URL: `/user/contacts?page=0&size=10&sortBy=name&direction=asc`
- 2 Controller: `Page pageContact = contactService.getByUser(user, page, size, sortBy, direction)`
- 3 `PageRequest.of(page, size, Sort.by(sortBy))` passed to `JpaRepository`
- 4 JPA auto-generates LIMIT, OFFSET, ORDER BY in SQL
- 5 Thymeleaf renders pagination bar using `getTotalPages()` and `getNumber()`
- 6 Null-safe: empty search field defaults to name-based search to prevent crashes

Feature 04

Custom Star-Rating & Feedback System

THE PROBLEM

Standard HTML rating inputs are plain and hard to validate elegantly.

- 1 5 star buttons each with `data-val 1-5`
- 2 JS click listener updates hidden input (`#rating-input`) and CSS (`text-amber-400`)
- 3 Backend: `@Valid @ModelAttribute` `FeedbackForm` with `@Min(1) @Max(5)` constraints
- 4 `BindingResult` captures validation errors; form re-shown on failure
- 5 Thymeleaf history: `#numbers.sequence(1,5)` renders ★ vs ■ dynamically

SECTION 05

AWS Deployment Architecture

01	Platform	AWS Elastic Beanstalk	Java SE platform with Amazon Corretto 21. Managed environment — auto handles load balancer, health monitoring, rolling deploys.
02	Reverse Proxy	Nginx (auto-configured)	Beanstalk spins up Nginx automatically. Routes external port 80/443 traffic → internal port 5000.
03	Spring Port	SERVER_PORT=5000	Set via AWS environment variables. Custom Procfile contains: web: java -jar scm2.0.jar
04	Database	Amazon RDS MySQL	External managed MySQL. Connection configured via env var placeholders: spring.datasource.url=jdbc:mysql://\${MYSQL_HOST}:\${MYSQL_PORT}/\${MYSQL_DB}
05	Credentials	Environment Variables	Passwords never hardcoded. MYSQL_HOST, MYSQL_PORT, MYSQL_DB, MYSQL_PASSWORD injected via AWS Console → safe for public Git repos.
06	Build & Deploy	./mvnw clean package	Packages app to .jar. Zipped with Procfile and uploaded directly to Elastic Beanstalk dashboard.

SECTION 06

Top 20 Interview Questions & Answers

Q
1**What is Spring Boot and how is it different from standard Spring Framework?**

Spring Framework requires manual XML/Java configurations, server setup, and dependency wiring. Spring Boot provides Starter dependencies (bundled configs), Auto-configuration (beans configured from classpath), and an Embedded Tomcat server — enabling production-ready apps with minimal setup.

Q
2**Explain the security filter chain implementation in your SecurityConfig.java.**

We configure a SecurityFilterChain bean using authorizeHttpRequests. Routes under /user/** require authentication; all others are public. It also wires our custom login page, /authenticate processing route, OAuth2 SSO, failure handler, and logout handler.

Q
3**Why did you disable CSRF in your configuration?**

In development, CSRF was disabled (AbstractHttpConfigurer::disable) to simplify POST form testing without injecting tokens manually. In production with Thymeleaf forms, CSRF should remain enabled — Thymeleaf auto-embeds the hidden token field when using th:action.

Q
4**How does the OAuth2 authentication flow work end-to-end?**

1) User clicks Login with Google → redirected to /oauth2/authorization/google. 2) Google Authorization endpoint shown. 3) User approves → Google redirects with auth code. 4) Spring exchanges code for Access Token and retrieves claims. 5) OAuthSuccessHandler extracts email/name/picture, checks DB, registers if new, logs them in, redirects to /user/profile.

Q
5**How do you handle image uploads? Why not save them on server disk?**

We use the Cloudinary SDK. MultipartFile bytes are uploaded via cloudinary.uploader().upload(...). We avoid server disk storage because cloud environments are ephemeral — autoscaling replaces instances and destroys local files. Cloudinary provides persistent CDN-hosted storage.

Q
6**How does Spring Data JPA perform pagination and sorting?**

A Pageable object (`PageRequest.of(page, size, Sort.by(sortBy))`) is passed to the repository. Spring Data JPA auto-generates LIMIT, OFFSET, and ORDER BY in SQL. Returns a Page with data subset and metadata like `getTotalPages()` and `getNumber()`.

Q
7**Explain the User-Contact relationship and how cascading works.**

It is `@OneToMany` on User side and `@ManyToOne` on Contact side. We use `cascade = CascadeType.ALL` and `orphanRemoval = true`. Deleting a user auto-deletes all their contacts. Removing a contact from the user's list auto-deletes it from the DB.

Q
8**What is the purpose of CommandLineRunner in Application.java?**

CommandLineRunner executes code once after Spring context initialization. We use it to run JdbcTemplate SQL to ensure nullable columns exist, and to auto-insert a default admin account (`admin@gmail.com`) on first startup.

Q
9**How do you send emails? What configurations were required?**

Spring Boot Starter Mail injects JavaMailSender. `application-dev.properties` configures `smtp.gmail.com` host, port 587, TLS flags, and app password. `EmailServiceImpl` builds `MimeMessage` using `MimeMessageHelper` for structured headers like Reply-To.

Q
10**How did you implement client-side Excel exporting? Why not backend?**

Integrated `@linways/table-to-excel` via CDN. A click event triggers `TableToExcel.convert(document.getElementById('contact-table'), { name: 'contacts.xlsx' })`. Client-side eliminates backend CPU load, avoids Apache POI complexity, and downloads the file instantly.

Q
11**What is the difference between Lazy and Eager fetching? Which did you use?**

EAGER loads child entities immediately alongside the parent. LAZY loads them only when the getter is called. We used LAZY for `User→Contacts` (avoid loading all contacts on profile fetch) and EAGER for `Contact→SocialLinks` (small in size, always shown together).

Q
12**How are form errors validated and shown to users?**

JSR-380 annotations (`@NotBlank`, `@Email`, `@Size`) on form beans like `ContactForm`. Controllers use `@Valid` and `BindingResult`. If `result.hasErrors()` is true, we return the user to the form page. Thymeleaf uses `th:if="${#fields.hasErrors('fieldName')}"` to render field-level errors.

Q
13**What does `createDatabaseIfNotExist=true` do in the JDBC URL?**

Instructs the MySQL JDBC driver to check if the target database exists on the host. If not, it executes a `CREATE DATABASE` statement before establishing the connection pool — useful for automatic schema creation on first AWS RDS startup.

Q
14**How did you handle null values in Contact Search?**

In `ContactController.java`, if the search category field is empty, the controller defaults to searching by 'name'. This null-safe check prevents sending null queries to the repository, which would cause database errors or Thymeleaf template evaluation crashes.

Q
15**What is Lombok? How does it save development time?**

Lombok is an annotation-based library generating boilerplate at compile time. `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`, and `@Builder` annotations on entities keep the codebase clean — no manual getter/setter methods needed.

Q
16**How did you manage database credentials safely on AWS?**

Used Spring's `${VAR_NAME:default}` placeholders in `application.properties`. On AWS Elastic Beanstalk, we set `MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_DB`, `MYSQL_PASSWORD` via the AWS Console. Spring Boot reads them at startup, keeping secrets out of Git repos.

Q
17**What is Spring Actuator?**

Actuator provides built-in HTTP endpoints like `/actuator/health`, `/actuator/info`, and `/actuator/metrics` to monitor application health, database connection status, and running configurations in production without code changes.

Q
18**What is the purpose of git_push.bat?**

A Windows batch script automating the Git workflow: shows git status, prompts for commit message, runs git add ., commits, and pushes to remote. Prevents manual errors and speeds up the release cycle.

Q
19**How would you debug a 'Field doesn't have a default value' error?**

1) Inspect the MySQL schema for NOT NULL columns without defaults. 2) Alter offending columns to be nullable (as done in Application.java via JdbcTemplate). 3) Ensure entity values are initialized programmatically before calling .save().

Q
20**If SCM 2.0 scales to 1 Million users, how would you modify the architecture?**

1) Decouple frontend: REST API + React/Next.js on Vercel/S3. 2) Database: RDS read-replicas for searches + Redis cache for profiles. 3) Load balancing: Stateless Spring Boot containers on AWS ECS behind Application Load Balancer with auto-scaling groups.